K-Nearest Neighbors (KNN) — Simple Overview

➢ Introduction

- **KNN** = Lazy learning algorithm.
- Works by looking at **nearest neighbors** to classify or predict a data point.

➢ KNN Intuition

- Imagine living in a neighborhood.
- To classify a new person, look at the **k closest neighbors** → majority vote decides the class.
- Distance metrics: Euclidean, Manhattan, Minkowski.

➢ How to Select K?

- Small k → sensitive to noise, may overfit.
- Large k → smoother decision boundary, may underfit.
- Common approach: use **odd k** (to avoid ties), test with **cross-validation**.

➢ Decision Surface

- Visualization of how KNN divides feature space.
- Boundary changes based on **k** value.

➢ Overfitting & Underfitting in KNN

- **Overfitting** → very small k (e.g., k=1).
- **Underfitting** → very large k (e.g., k=n).
- Balance is needed!

➢ Limitations of KNN

- **Computationally expensive** → must calculate distances to all points at prediction time.
- Doesn't work well on **high-dimensional data** (curse of dimensionality).
- Sensitive to **irrelevant features** and **scaling**.

➢ Outro

KNN is **simple, intuitive, and powerful** for small datasets but not ideal for **large-scale or high-dimensional problems**.

➢ Quick Takeaway

- KNN = Classifies by majority vote among k nearest neighbors.
- Choosing **k** is crucial (cross-validation helps).
- Works well for small datasets with low dimensions.
- Needs **feature scaling** and can be **slow for large data**.
